C++ Classes

C++ Interlude 1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Points

- Header (.h) files in C++ provide a mechanism to partially separate the design of a class from the implementation in the source , or implementation (.cpp), file.
- The header file must also contain a description of the data fields for the class and any private methods used by the class
- Elements of the class
  - Private data fields // good practice
    - restricts access to the data field to just the class in which it is defined
    - Typically, provide methods—such as setItem (setters) and getItem (getters)— to access the data fields
    - Class controls how and whether other classes can access the data fields
  - If a derived class might need to access or modify the data fields of the base class, you should still make the data fields private but provide protected methods so that any derived classes can access or modify the data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Constructors

- Elements of the class (cont.)
  - Constructors
    - Allocates memory for new instances of a class
    - Can initialize the object's data to specified values
    - A typical class has several constructors
      - default constructor initializes data fields to values that the class implementation chooses
      - Other constructors have parameters. These parameterized constructors initialize data fields to values chosen by the client but approved by the constructor.
    - Have the same name as the class
    - Have no return type—not even void—and cannot use return to return a value
    - The compiler decides which constructor to call by matching the argument list supplied by the client with the parameters of the available constructors. A match occurs when the arguments and parameters correspond in number, data type, and order

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Destructors

- Elements of the class (cont.)
  - Destructors
    - destroys an instance of a class when the object's lifetime ends
    - A class has one destructor
    - For many classes, you can omit the destructor. In such cases, the compiler generates a destructor for you.
    - If the class itself creates an object on the free store (heap) by using the new operator, it is a safe and secure programming practice to implement a destructor to ensure that the memory for that object is freed (cover in Interlude 2)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Methods

- Elements of the class (cont.)
  - Methods
    - Implement the algorithms that solve a problem
    - A method conveys three important pieces of information to a client wishing to use our class:
      - method name,
      - number and types of its parameters, and
      - method's return type.
    - methods a client can use are in the public section of the class declaration

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Getters/Setters

- Elements of the class (cont.)
  - An accessor method in a class (getters)
    - Accesses, or gets, the value of a data field
    - Name often begins with the word get
    - Declaring all accessor methods as const is good practice
      - ItemType getItem() const;
    - The const keyword signals to the compiler that the method does not change the data fields of the object
  - A mutator method in a class (setters)
    - Changes the value of a data field
    - Often, the name of a mutator method begins with the word set
    - Cannot be declared as const, because they must initialize the data fields of an object.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Preprocessor directives

- Elements of the class (cont.)
  - Preprocessor directives
    - Lines included in the code of programs preceded by a hash sign (#)
    - Not program statements but directives for the preprocessor
    - The preprocessor examines the code before actual compilation of code begins and resolves all these directives before any code is actually generated by regular statements, e.g.:
      - #define TABLE_SIZE 100
      - int table1[TABLE_SIZE] // equivalent to int table1[100]
      - #undef TABLE_SIZE
      - #define TABLE_SIZE 200
      - int table2[TABLE_SIZE] // equivalent to int table2[200]

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Preprocessor directives (cont.)

- Elements of the class (cont.)
  - Preprocessor directives (cont.)
    - Provides a way to prevent the compiler from reading the class definition a second time, if class definition is included multiple times:
      - #ifndef PLAIN_BOX // If PLAIN_BOX is not defined, then ...”
      - #define PLAIN_BOX // defines the name PLAIN_BOX
      - …
      - endif PLAIN_BOX

If the compiler had not defined the name PLAIN_BOX, it would process the code that follows until it reached the #endif directive at the end of the file.

If another file includes the class definition of PlainBox, the name PLAIN_BOX will already have been defined. That file’s #ifndef directive will cause the preprocessor to skip any of the code that follows, so the compiler will not see the same class definition more than once.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Templates

- Templates are a way to allow functions and classes to use the same code for many different data types.

```
#include <string>
#include <iostream>

template <class MyType>    // declares the template
class MyClass {
public:
   MyType first;
   MyType second;

   MyClass(MyType f, MyType s) {
      first = f;
      second = s;
   }
};

int main() {
   // Use MyClass with strings
   MyClass<std::string> stringObj("Hello", "World");
   std::cout << stringObj.first << " " << stringObj.second << "\n";

   // Use MyClass with integers
   MyClass<int> intObj(4, 5);
   std::cout << intObj.first << " " << intObj.second << "\n";

   return 0;
}
```

class MyType is a template parameter that makes MyType possible to represent any data type

creates a MyClass object where MyType means string

creates a MyClass object where MyType means int

https://www.w3schools.com/cpp/cpp_templates.asp

C++ Inheritance

- Inheritance allows us to reuse and extend work we have already completed and tested.
- You can think of a base class as a parent class.
  - base class = superclass = parent class
- You can think of a derived class as a child class
  - derived class = subclass = child class
- Any instance of the derived class is also considered to be an instance of the base class and can be used in a program anywhere that an instance of the base class can be used
- When the keyword public is used with the base class, any of the publicly defined methods or data fields in the base class can be used by instances of the derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ derived class

- A derived class
  - has the data fields and methods of the base class in addition to the members it defines,
  - it can access only publicly defined members from the base class
  - can also revise any inherited public method.
  - contains copies of inherited data fields,
  - does not contain a copy of the code for inherited methods
  - does not inherit constructors from parent class, but it can use it or other parent method to initialize the private data fields

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Inheritance

LISTING C1-5 Template header file for the class ToyBox

```
/** @file ToyBox.h */

#ifndef _TOY_BOX
#define _TOY_BOX
#include "PlainBox.h"
enum Color {BLACK, RED, BLUE, GREEN, YELLOW, WHITE};

template<class ItemType>
class ToyBox : public PlainBox<ItemType>
{
private:
   Color boxColor;

public:
   ToyBox();
   ToyBox(const Color& theColor);
   ToyBox(const ItemType& theItem, const Color& theColor);
   Color getColor() const;
}; // end ToyBox
#include "ToyBox.cpp"
#endif
```

LISTING C1-6 Implementation file for the class ToyBox

```
/** @file ToyBox.cpp */

template<class ItemType>
ToyBox<ItemType>::ToyBox()
{
   PlainBox<ItemType>();
   boxColor = BLACK;
}  // end default constructor

template<class ItemType>
ToyBox<ItemType>::ToyBox(const Color& theColor)
{
   PlainBox<ItemType>();
   boxColor = theColor;
}  // end constructor

template<class ItemType>
ToyBox<ItemType>::ToyBox(const ItemType& theItem,
                         const Color& theColor)
{
   PlainBox<ItemType>();
   PlainBox<ItemType>::setItem(theItem);
   boxColor = theColor;
}  // end constructor

template<class ItemType>
Color ToyBox<ItemType>::getColor() const
{
   return boxColor;
}  // end getColor
```

ToyBox 1) defines constructors and specific new method, 2)inherits data field item, and methods setItem, getItem from PlainBox class

We can create and use a ToyBox object as follows:

```
string favoriteToy = "Jack-in-the-Box";
ToyBox<string> myToyCase(favoriteToy, RED); // A red toy box
string oldToy = myToyCase.getItem();      // oldToy is a Jack-in-the-Box
favoriteToy = "Spinning Top";
myToyCase.setItem(favoriteToy);     // myToyCase now holds a Spinning Top
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overriding Base-Class Methods

- You can add as many new members to derived class as desired
- You can redefine inherited methods
  - Called overriding a base-class method.
- A method overrides a base-class method when
  - The two methods have the same name and parameter declarations
- Precede a method name with the base-class namespace indicator to tell the compiler to use the base-class version of the method
- Without a namespace indicator in front of the method name, the compiler assumes that the programmer is referring to the current class and inserts code to call that version of the method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods, Abstract Classes

- Using keyword **virtual** in front of the prototype
  - Tells the C++ compiler that the code this method executes is determined at runtime
  - Allows an application using our class to take advantage of polymorphism when the methods are invoked
- Pure virtual method
  - Virtual method that has no implementation
- Abstract class
  - Has at least one pure virtual method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods, Abstract Classes

```
/** @file BoxInterface.h */

#ifndef BOX_INTERFACE_
#define BOX_INTERFACE_

template <class ItemType>
class  BoxInterface
{
public:
   virtual void  setItem(const ItemType& theItem) = 0;
   virtual ItemType getItem() const = 0;
   virtual ~BoxInterface() {} // C++ Interlude 2 explains virtual destructors
}; // end BoxInterface
#endif
```

LISTING C1-9 An abstract class that is an interface for the ADT box

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Interlude 1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Interlude 1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

- Consider a video game where a character carries three types of boxes
  - Plain box
  - Toy box
  - Magic box
- Plain box design
  - Get and Set public methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
/** @file PlainBox.h */

#ifndef PLAIN_BOX_
#define PLAIN_BOX_

// Set the type of data stored in the box
typedef double ItemType;
// Declaration for the class PlainBox
class PlainBox
{
private:
   // Data field
   ItemType item;

public:
```

LISTING C1-1 The header file for the class PlainBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
public:
   // Default constructor
   PlainBox();

   // Parameterized constructor
   PlainBox(const ItemType& theItem);

   // Method to change the value of the data field
   void setItem(const ItemType& theItem);

   // Method to get the value of the data field
   ItemType getItem() const;
}; // end PlainBox
#endif
```

LISTING C1-1 The header file for the class PlainBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
/** @file PlainBox.cpp */

#include "PlainBox.h"

PlainBox::PlainBox()
{
}  // end default constructor
PlainBox::PlainBox(const ItemType& theItem)
{
   item = theItem;
}  // end constructor
void PlainBox::setItem(const ItemType& theItem)
{
   item = theItem;
}  // end setItem

ItemType PlainBox::getItem() const
{
   return item;
}  // end getItem
```

LISTING C1-2 Implementation file for the PlainBox class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
/** @file PlainBox.h */

#ifndef PLAIN_BOX_
#define PLAIN_BOX_

template<class ItemType> // Indicates this is a template definition

// Declaration for the class PlainBox
class  PlainBox
{
private:
   // Data field
   ItemType item;
public:
   // Default constructor
   PlainBox();
```

LISTING C1-3 Template header file for the PlainBox class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
    PlainBox();

    // Parameterized constructor
    PlainBox(const ItemType& theItem);

    // Mutator method that can change the value of the data field
    void setItem(const ItemType& theItem);

    // Accessor method to get the value of the data field
    ItemType getItem() const;
};  // end PlainBox

#include "PlainBox.cpp" // Include the implementation file
#endif
```

LISTING C1-3 Template header file for the PlainBox class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
/** @file PlainBox.cpp */
#include "PlainBox.h"

template<class ItemType>
PlainBox<ItemType>::PlainBox()
{
}  // end default constructor

template<class ItemType>
PlainBox<ItemType>::PlainBox(const ItemType& theItem) : item(theItem)
{
}  // end constructor
```

LISTING C1-4 Implementation file for the PlainBox template class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Problem to Solve

```
template<class ItemType>
void PlainBox<ItemType>::setItem(const ItemType& theItem)
{
   item = theItem;
}  // end setItem

template<class ItemType>
ItemType PlainBox<ItemType>::getItem() const
{
   return item;
}  // end getItem
```

LISTING C1-4 Implementation file for the PlainBox template class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Base Classes and Derived Classes

- Use PlainBox as a base class , or superclass
- The ToyBox class is the derived class , or subclass , of the PlainBox
- Derived class inherits
  - All the members of its base class,
  - (Except the constructors and destructor)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Base Classes and Derived Classes

```
/** @file ToyBox.h */

#ifndef TOY_BOX_
#define TOY_BOX_
#include "PlainBox.h"

enum Color {BLACK, RED, BLUE, GREEN, YELLOW, WHITE};

template<class ItemType>
class ToyBox : public PlainBox<ItemType>
{
private:
   Color boxColor;

public:
   ToyBox();
   ToyBox(const Color& theColor);
   ToyBox(const ItemType& theItem, const Color& theColor);
   Color getColor() const;
}; // end ToyBox
#include "ToyBox.cpp"
#endif
```

LISTING C1-5 Template header file for the class ToyBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Base Classes and Derived Classes

```
/** @file ToyBox.cpp */

#include "ToyBox.h"

template<class ItemType>
ToyBox<ItemType>::ToyBox() : boxColor(BLACK)
{
}  // end default constructor

template<class ItemType>
ToyBox<ItemType>::ToyBox(const Color& theColor) : boxColor(theColor)
{
}  // end constructor

template<class ItemType>
ToyBox<ItemType>::ToyBox(const ItemType& theItem, const Color& theColor)
                        : PlainBox<ItemType>(theItem), boxColor(theColor)
{
}  // end constructor

template<class  ItemType>
Color ToyBox<ItemType>::getColor() const
{
   return boxColor;
}  // end getColor
```

LISTING C1-6 Implementation file for the class ToyBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overriding Base-Class Methods

```
/** @file MagicBox.h */

#ifndef MAGIC_BOX_
#define MAGIC_BOX_
#include "PlainBox.h"

template<class ItemType>
class MagicBox: public PlainBox<ItemType>
{
private:
   bool firstItemStored;

public:
   MagicBox();
   MagicBox(const ItemType& theItem);
   void setItem(const ItemType& theItem);
}; // end MagicBox
#include "MagicBox.cpp"
#endif
```

LISTING C1-7 Header file for the class MagicBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overriding Base-Class Methods

```cpp
/** @file MagicBox.cpp */
#include "MagicBox.h"
template<class ItemType>
MagicBox<ItemType>::MagicBox(): firstItemStored(false)
{
   // PlainBox constructor is called implicitly.
   // Box has no magic initially
}  // end default constructor

template<class ItemType>
MagicBox<ItemType>::MagicBox(const ItemType& theItem): firstItemStored(false)
{
   // Box has no magic initially
   setItem(theItem); // Calls MagicBox version of setItem
   // Box has magic now
}  // end constructor

template<class ItemType>
void MagicBox<ItemType>::setItem(const ItemType& theItem)
{
   if (!firstItemStored)
   {
      PlainBox<ItemType>::setItem(theItem);
      firstItemStored = true; // Box has magic now
   }  // end if
}  // end setItem
```

LISTING C1-8 Implementation file for the class MagicBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved